University of Science and Technology of Oran – Mohamed Boudiaf
Faculty of Mathematics and Informatics
Department of Computer Science

TP 0 : Basics

Checking the basics: loops, linked lists and searching

Mohammed Said BELAID
said.belaid@univ-usto.dz

13/10/2026

Objectives

This first TP checks that you master the basics expected in this course:

- writing loops and conditions,
- implementing and modifying a simple data structure,
- comparing several solutions to the same problem,
- reasoning about time complexity.

Exercise 1: FizzBuzz

1. Write a program that prints the numbers from 1 to 100, with the following rules:
   - for multiples of 3, print `Fizz` instead of the number,
   - for multiples of 5, print `Buzz` instead of the number,
   - for multiples of both 3 and 5, print `FizzBuzz`.

   Expected beginning of the output: `1 2 Fizz 4 Buzz Fizz 7 8 Fizz Buzz 11 Fizz 13 14 FizzBuzz 16 ...`

2. Write a function `fizzbuzz(n)` that prints the sequence from 1 to `n`. The value of `n` is read from the user.

3. We now add a new rule: for multiples of 7, print `Bazz`. A number that is a multiple of several values combines the words in order (21 → `FizzBazz`, 35 → `BuzzBazz`, 105 → `FizzBuzzBazz`).
   - Modify your program to support this rule.

Exercise 2: Linked list

1. Implement a singly linked list of integers. Each node contains a value and a reference to the next node. The list must provide:

Operation	Description
`addFirst(x)`	insert `x` at the beginning of the list
`addLast(x)`	insert `x` at the end of the list
`contains(x)`	return true if `x` is in the list
`remove(x)`	remove the first occurrence of `x`, if present
`print()`	display the elements of the list

Test every operation, including on an empty list and on a list with a single element.

2. Add a method `size()` that returns the number of elements by traversing the list.
   - What is the time complexity of `size()`?
   - A program calls `size()` after every insertion while building a list of n elements. What is the total cost?

3. Modify your list so that `size()` runs in $O(1)$.
   - Which operations must update it?
   - Give the time complexity of each operation of your list.

4. **(Bonus)** Make `addLast(x)` run in $O(1)$.

Exercise 3: Find the duplicate

Given an array of n integers, write a function `hasDuplicate(a)` that returns true if at least one value appears more than once, and false otherwise.

```
hasDuplicate([4, 1, 7, 3, 1, 9])  ->  true    (1 appears twice)
hasDuplicate([4, 1, 7, 3, 8, 9])  ->  false
hasDuplicate([])                  ->  false
```

1. **Naive solution.** Write a first version that compares every pair of elements.
   - What is its time complexity? Justify.
   - How many comparisons are done in the worst case for $n = 10^6$?

2. **A better solution.** Write a second version that is faster than the naive one.

   Hint: in a sorted array, where are the copies of a value?
   - What is its time complexity?